

EEE3096S: Practical 1A Demo
Phila Haanyama (HNYPHI001) and Lulama Lingela (LNGLUL002)
File: Practical_1A/Core/Src/main.c

```
/* USER CODE BEGIN Header */
/**
 * *****
 * @file           : main.c
 * @brief          : Main program body
 * *****
 * @attention
 *
 * Copyright (c) 2023 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 * *****
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */
#include <stdint.h>
#include <stdlib.h>
#include "stm32f0xx.h"
/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/
TIM_HandleTypeDef htim16;

/* USER CODE BEGIN PV */
// TODO: Define input variables

// Current mode: 0 = off, 1 = back/forth, 2 = inverse back/forth, 3 = sparkle
uint8_t led_mode = 0;
volatile int8_t dir = 1; // 1 = to the left, -1 = to the right
uint8_t current_led = 0;
uint8_t direction = 1; // 1 = forward, 0 = backward
uint8_t sparkle_state = 0;
// when state = 0: turns on random pattern, sets hold delay
// when state = 1: waits randomly 100-1500ms
// when state = 2: turns off 1 LED at a time, random delay

uint8_t sparkle_pattern = 0;
uint32_t timer_delay = 1000; // default 1s
```

```

uint32_t sparkle_hold_ticks = 0;      // time to hold the sparkle pattern
uint32_t sparkle_off_delay = 0;      // delay between turning off LEDs
uint32_t sparkle_tick_counter = 0;    // tracks time across interrupts

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM16_Init(void);
/* USER CODE BEGIN PFP */
void TIM16_IRQHandler(void);
/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */
    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the Systick.
    */
    HAL_Init();

    // ADDED TO MAKE RANDOM PATTERN FOR SPARKLE
    srand(HAL_GetTick()); // seed random generator with system tick

    /* USER CODE BEGIN Init */
    /* USER CODE END Init */

    /* Configure the system clock */
    SystemClock_Config();

    /* USER CODE BEGIN SysInit */
    /* USER CODE END SysInit */

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_TIM16_Init();
    /* USER CODE BEGIN 2 */

    // TODO: Start timer TIM16
    HAL_TIM_Base_Start_IT(&htim16);

    /* USER CODE END 2 */

    /* Infinite loop */
    /* USER CODE BEGIN WHILE */
    while (1)
    {
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */

```

```

// TODO: Check pushbuttons to change timer delay

    if (LL_GPIO_IsInputPinSet(GPIOA, Button0_Pin) == 0) { // Active low
        if (timer_delay == 1000)
            timer_delay = 500;
        else
            timer_delay = 1000;

        __HAL_TIM_SET_AUTORELOAD(&htim16, timer_delay);
        HAL_Delay(300); // debounce
    }

}
/* USER CODE END 3 */
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
    LL_FLASH_SetLatency(LL_FLASH_LATENCY_0);
    while(LL_FLASH_GetLatency() != LL_FLASH_LATENCY_0)
    {
    }
    LL_RCC_HSI_Enable();

    /* Wait till HSI is ready */
    while(LL_RCC_HSI_IsReady() != 1)
    {

    }
    LL_RCC_HSI_SetCalibTrimming(16);
    LL_RCC_SetAHBPrescaler(LL_RCC_SYSCLK_DIV_1);
    LL_RCC_SetAPB1Prescaler(LL_RCC_APB1_DIV_1);
    LL_RCC_SetSysClkSource(LL_RCC_SYS_CLKSOURCE_HSI);

    /* Wait till System clock is ready */
    while(LL_RCC_GetSysClkSource() != LL_RCC_SYS_CLKSOURCE_STATUS_HSI)
    {

    }
    LL_SetSystemCoreClock(8000000);

    /* Update the time base */
    if (HAL_InitTick (TICK_INT_PRIORITY) != HAL_OK)
    {
        Error_Handler();
    }
}

/**
 * @brief TIM16 Initialization Function
 * @param None
 * @retval None
 */
static void MX_TIM16_Init(void)
{

    /* USER CODE BEGIN TIM16_Init 0 */

    /* USER CODE END TIM16_Init 0 */

```

```

/* USER CODE BEGIN TIM16_Init 1 */

/* USER CODE END TIM16_Init 1 */
htim16.Instance = TIM16;
htim16.Init.Prescaler = 8000-1;
htim16.Init.CounterMode = TIM_COUNTERMODE_UP;
htim16.Init.Period = 1000-1;
htim16.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim16.Init.RepetitionCounter = 0;
htim16.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&htim16) != HAL_OK)
{
    Error_Handler();
}
/* USER CODE BEGIN TIM16_Init 2 */
NVIC_EnableIRQ(TIM16_IRQn);
/* USER CODE END TIM16_Init 2 */

}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    LL_GPIO_InitTypeDef GPIO_InitStruct = {0};
/* USER CODE BEGIN MX_GPIO_Init_1 */
/* USER CODE END MX_GPIO_Init_1 */

    /* GPIO Ports Clock Enable */
    LL_AHB1_GRP1_EnableClock(LL_AHB1_GRP1_PERIPH_GPIOF);
    LL_AHB1_GRP1_EnableClock(LL_AHB1_GRP1_PERIPH_GPIOA);
    LL_AHB1_GRP1_EnableClock(LL_AHB1_GRP1_PERIPH_GPIOB);

    /**/
    LL_GPIO_ResetOutputPin(LED0_GPIO_Port, LED0_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED1_GPIO_Port, LED1_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED2_GPIO_Port, LED2_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED3_GPIO_Port, LED3_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED4_GPIO_Port, LED4_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED5_GPIO_Port, LED5_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED6_GPIO_Port, LED6_Pin);

    /**/
    LL_GPIO_ResetOutputPin(LED7_GPIO_Port, LED7_Pin);

    /**/
    GPIO_InitStruct.Pin = Button0_Pin;
    GPIO_InitStruct.Mode = LL_GPIO_MODE_INPUT;
    GPIO_InitStruct.Pull = LL_GPIO_PULL_UP;
    LL_GPIO_Init(Button0_GPIO_Port, &GPIO_InitStruct);

```

```

/**/
GPIO_InitStruct.Pin = Button1_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = LL_GPIO_PULL_UP;
LL_GPIO_Init(Button1_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = Button2_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = LL_GPIO_PULL_UP;
LL_GPIO_Init(Button2_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = Button3_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_INPUT;
GPIO_InitStruct.Pull = LL_GPIO_PULL_UP;
LL_GPIO_Init(Button3_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED0_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED0_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED1_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED1_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED2_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED2_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED3_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED3_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED4_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED4_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED5_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED5_GPIO_Port, &GPIO_InitStruct);

```

```

/**/
GPIO_InitStruct.Pin = LED6_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED6_GPIO_Port, &GPIO_InitStruct);

/**/
GPIO_InitStruct.Pin = LED7_Pin;
GPIO_InitStruct.Mode = LL_GPIO_MODE_OUTPUT;
GPIO_InitStruct.Speed = LL_GPIO_SPEED_FREQ_LOW;
GPIO_InitStruct.OutputType = LL_GPIO_OUTPUT_PUSHPULL;
GPIO_InitStruct.Pull = LL_GPIO_PULL_NO;
LL_GPIO_Init(LED7_GPIO_Port, &GPIO_InitStruct);

/* USER CODE BEGIN MX_GPIO_Init_2 */
/* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */
void TIM16_IRQHandler(void)
{
    // Acknowledge interrupt
    HAL_TIM_IRQHandler(&htim16);

    // TODO: Change LED pattern

    // Check for mode changes (active low)
    if (LL_GPIO_IsInputPinSet(GPIOA, Button1_Pin) == 0) {
        led_mode = 1;
        current_led = 0;
        direction = 1;
    }
    else if (LL_GPIO_IsInputPinSet(GPIOA, Button2_Pin) == 0) {
        led_mode = 2;
        current_led = 0;
        direction = 1;
    }
    else if (LL_GPIO_IsInputPinSet(GPIOA, Button3_Pin) == 0) {
        led_mode = 3;
        sparkle_state = 0;
    }

    switch (led_mode) {
        case 1: // mode 1: back/forth
            LL_GPIO_ResetOutputPin(GPIOB, LED0_Pin | LED1_Pin | LED2_Pin |
LED3_Pin | LED4_Pin | LED5_Pin | LED6_Pin | LED7_Pin);
            switch(current_led){
                case 0: LL_GPIO_SetOutputPin(GPIOB, LED0_Pin);
                        break;

                case 1: LL_GPIO_SetOutputPin(GPIOB, LED1_Pin);
                        break;

                case 2: LL_GPIO_SetOutputPin(GPIOB, LED2_Pin);
                        break;

                case 3: LL_GPIO_SetOutputPin(GPIOB, LED3_Pin);
                        break;

                case 4: LL_GPIO_SetOutputPin(GPIOB, LED4_Pin);
                        break;

                case 5: LL_GPIO_SetOutputPin(GPIOB, LED5_Pin);
                        break;
            }

```

```

        case 6: LL_GPIO_SetOutputPin(GPIOB, LED6_Pin);
                break;

        case 7: LL_GPIO_SetOutputPin(GPIOB, LED7_Pin);
                break;
    }

    if (current_led == 7){
        dir = -1;
    }
    else if (current_led == 0){
        dir = 1;
    }

    current_led += dir;

    break;

case 2: // mode 2: inverse back/forth
    LL_GPIO_SetOutputPin(GPIOB, LED0_Pin | LED1_Pin | LED2_Pin |
LED3_Pin |
                                LED4_Pin | LED5_Pin | LED6_Pin |
LED7_Pin);

    switch (current_led) {
        case 0: LL_GPIO_ResetOutputPin(GPIOB, LED0_Pin); break;
        case 1: LL_GPIO_ResetOutputPin(GPIOB, LED1_Pin); break;
        case 2: LL_GPIO_ResetOutputPin(GPIOB, LED2_Pin); break;
        case 3: LL_GPIO_ResetOutputPin(GPIOB, LED3_Pin); break;
        case 4: LL_GPIO_ResetOutputPin(GPIOB, LED4_Pin); break;
        case 5: LL_GPIO_ResetOutputPin(GPIOB, LED5_Pin); break;
        case 6: LL_GPIO_ResetOutputPin(GPIOB, LED6_Pin); break;
        case 7: LL_GPIO_ResetOutputPin(GPIOB, LED7_Pin); break;
    }

    if (direction)
        current_led++;
    else
        current_led--;

    if (current_led >= 7) direction = 0;
    if (current_led <= 0) direction = 1;
    break;

case 3: // mode 3: Sparkle
    if (sparkle_state == 0) {
        // generate random LED pattern
        sparkle_pattern = rand() % 256;

        if (sparkle_pattern & 0x01) LL_GPIO_SetOutputPin(GPIOB,
LED0_Pin);
        if (sparkle_pattern & 0x02) LL_GPIO_SetOutputPin(GPIOB,
LED1_Pin);
        if (sparkle_pattern & 0x04) LL_GPIO_SetOutputPin(GPIOB,
LED2_Pin);
        if (sparkle_pattern & 0x08) LL_GPIO_SetOutputPin(GPIOB,
LED3_Pin);
        if (sparkle_pattern & 0x10) LL_GPIO_SetOutputPin(GPIOB,
LED4_Pin);
        if (sparkle_pattern & 0x20) LL_GPIO_SetOutputPin(GPIOB,
LED5_Pin);
        if (sparkle_pattern & 0x40) LL_GPIO_SetOutputPin(GPIOB,
LED6_Pin);
        if (sparkle_pattern & 0x80) LL_GPIO_SetOutputPin(GPIOB,
LED7_Pin);
    }
}

```

```

        // random hold time between 100-1500ms
        sparkle_hold_ticks = (rand() % 1400) + 100;
        sparkle_tick_counter = 0;

        sparkle_state = 1;
        current_led = 0; // start turning off from LED0
    } else if (sparkle_state == 1) {
        // hold sparkle pattern until timer reaches hold_ticks
        sparkle_tick_counter += timer_delay;
        if (sparkle_tick_counter >= sparkle_hold_ticks) {
            sparkle_state = 2;
            sparkle_tick_counter = 0;
            sparkle_off_delay = (rand() % 100) + 1; // first off
delay
        }
    } else if (sparkle_state == 2) {
        // turn off 1 LED at a time with delay
        sparkle_tick_counter += timer_delay;

        if (sparkle_tick_counter >= sparkle_off_delay) {
            sparkle_tick_counter = 0;

            // skip LEDs that are off
            while (current_led < 8 && !(sparkle_pattern & (1 <<
current_led)))) {
                current_led++;
            }

            if (current_led < 8) {
                switch (current_led) {
                    case 0: LL_GPIO_ResetOutputPin(GPIOB,
LED0_Pin); break;
                    case 1: LL_GPIO_ResetOutputPin(GPIOB,
LED1_Pin); break;
                    case 2: LL_GPIO_ResetOutputPin(GPIOB,
LED2_Pin); break;
                    case 3: LL_GPIO_ResetOutputPin(GPIOB,
LED3_Pin); break;
                    case 4: LL_GPIO_ResetOutputPin(GPIOB,
LED4_Pin); break;
                    case 5: LL_GPIO_ResetOutputPin(GPIOB,
LED5_Pin); break;
                    case 6: LL_GPIO_ResetOutputPin(GPIOB,
LED6_Pin); break;
                    case 7: LL_GPIO_ResetOutputPin(GPIOB,
LED7_Pin); break;
                }

                current_led++;
                sparkle_off_delay = (rand() % 100) + 1; // new
delay for next LED
            } else {
                // all LEDs handled -> restart
                sparkle_state = 0;
            }
        }
    }
    break;

default:
    // only turn off all LEDs when no mode active
    LL_GPIO_ResetOutputPin(GPIOB, LED0_Pin | LED1_Pin | LED2_Pin
| LED3_Pin |
                                LED4_Pin | LED5_Pin | LED6_Pin
| LED7_Pin);

```

```

        break;
    }
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line number
 * where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and line
    number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```